

OBN Codebase Health Check & Findings

Code-health review, findings, and remediation plan

Prepared for	Hicham El Khiati, Technical Director
Author	Abbas Rizvi
Date	27 June 2026
Subject	Independent code-health review of the OBN codebase: findings and a remediation plan
Review baseline	OBN origin/master, commit 8042c8d, version 2.3.12 (current shipped R&D tree)
Document	Part A — summary (leadership); Part B — technical findings (engineering); Part C — downstream-consumer (NMS / Zabbix) impact
Classification	Confidential — Internal
Revision	v2.0 (27 Jun 2026) — reframed as a code-health review and remediation plan; the replace-vs-improve comparison and all cost estimates removed (the team is proceeding with improvement). v1.x carried the decision framing.

Summary

OBN is fundamentally sound — structurally simple, no known dependency vulnerabilities, no injection exposure, and a real test suite.

Its defects reduce to a small number of recurring patterns with common fixes, addressed by the six-part hardening programme in §6. One item is urgent and should be actioned immediately: device passwords are stored in clear text in the source-code repository (finding S3).

Contents

Contents	2
Glossary	4
1. Summary	6
2. What we reviewed and how	6
3. The health of the code	7
4. Where the problems are	7
4.1 One systemic root cause.....	7
4.2 One real architectural weakness, and it is contained	8
4.3 What the audit found	8
5. Overall code-health assessment	8
6. Remediation plan	9
B1. Scope, method, baseline metrics	10
B2. Architecture.....	10
B2.1 The architectural weakness — the report layer.....	11
B2.2 The systemic root cause — None on SNMP failure.....	12
B3. Dependencies & security.....	12
B3.0 Findings S3–S7 (HIGH) — credential exposure cluster (plaintext, reused, shipped, world-readable, unrotated).....	12
B3.1 Finding S1 (MED–HIGH) — unauthenticated, GET-mutating REST API	13
B4. Bug & edge-case sweep (adversarially verified).....	14
B4.1 Confirmed findings (15)	14
B4.2 The defects are two systemic patterns, not 39 unrelated bugs.....	15
B4.3 Whole-platform spread (not DOSTO-only).....	15
B4.4 The untested-critical-path payoff	16
B4.5 Verification credibility	16
B5. Maintainability & tests.....	16
B6. Code-health scorecard	16
B7. Remediation programme (technical).....	16
B8. Central-module (high-blast-radius) audit.....	17
B10. Onboarding & prestaging fragility audit.....	17
B10.1 The single root cause	18
B10.2 Confirmed findings (19)	18
B10.3 The fix converges to three primitives.....	19
B10.4 Bearing on the review	20
B11. Vendor firmware-coupling fragility.....	20
B11.1 Where the coupling lives.....	20
B11.2 Remediation.....	21

B9. Caveats & artifacts	21
C1. Who consumes OBN data, and how	22
C2. Three classes of consumer-impacting defect (origin/master as-is)	22
C2.1 Dropped devices (most serious)	22
C2.2 Flap / spurious events (no debounce)	22
C2.3 Stale-suppression and wrong fields	23
C3. Confirmed consumer-impacting findings (8)	23
C4. Where each fix belongs	24

Glossary

Plain-language meaning of the terms used in this report. Part A (executive) is written to be read without these; they appear mainly in Parts B and C (engineering).

Term	Plain meaning
OBN	On-Board Networking — the tool that discovers, configures and monitors the network equipment on each train. The subject of this report.
NMS	Network Management System — the dashboard (used by ÖBB) that shows each train's devices, status and inventory. A consumer of OBN's data.
Zabbix	The monitoring system that polls devices and raises alarms. Also a consumer of OBN's data.
SNMP	The standard protocol OBN uses to talk to network devices (read status, push firmware/config).
MQTT	The lightweight messaging channel OBN uses to publish its results to NMS.
CCU	The on-train computer (jump-box) that runs OBN.
Consist / coach	A consist is a coupled train set; a coach is one car. OBN works out which device sits in which coach.
Report class / template	Per-customer logic (a “report class” in code) and per-fleet configuration (a “template”) that describe a given train's layout.
Firmware / configuration	The software image and settings running on a device; OBN checks and updates these.
Credential / plaintext	A password or key; “plaintext” means stored unencrypted, readable as-is.
Repository (repo)	The store holding the source code and configuration, with full change history.
.deb / package	The installable bundle of OBN that is deployed to each train computer.
None (null)	A “no value” result — what OBN gets when a device does not answer; mishandling it is the main source of crashes.
Cyclomatic complexity / maintainability index	Standard automated measures of how tangled / how maintainable code is (higher maintainability = better).
LOC	Lines of code — a size measure.
CVE	A publicly known software vulnerability; a “CVE scan” checks dependencies against the public list.
API / REST / GET / POST	A network interface other software calls; GET/POST are request types (GET should only read, POST should be used for actions).
Confirmed / needs-runtime / refuted	A confirmed defect is provable from the code; needs-runtime needs a live device to trigger; refuted was checked and dismissed.

Term	Plain meaning
Blast radius	How widely a single defect spreads — one device, one customer, or every train.

PART A — SUMMARY

1. Summary

This is an independent code-health review of **OBN** — On-Board Networking, the tool that automatically discovers, configures and monitors the network equipment on each train — covering the code as R&D currently ships it (version 2.3.12). It sets out the state of the code, the defects found, and a remediation plan (§6). The team is proceeding with improving OBN in place; this document is the basis for that work, not a decision about whether to do it.

The headline findings:

1. **The code is fundamentally sound.** OBN is structurally simple (every file rates maintainability grade A on the standard automated measure), has no known dependency vulnerabilities, no injection exposure, and a real test suite. The problems are fixable in place.
2. **The defects are a few patterns, not many separate problems.** A comprehensive, independently-verified audit found that the large majority of defects reduce to a small number of recurring patterns, each with one common fix. Fixing the pattern retires dozens of latent bugs at once. A follow-up review of how OBN brings a device on board — prompted by a product report that vendor firmware updates silently break device support — confirmed that the same fail-silent character runs through the whole onboarding and prestaging path, not just the report layer (§4.1, §B10, §B11).
3. **The platform is large and in active service.** A live check of our source-code repositories found about 49 customer fleets defined across 65 train-layout templates (operators across Austria, Germany, France, Spain, the Netherlands, Denmark, Norway, Luxembourg, Morocco, Australia and the UK — Eurostar, Renfe, Talgo, DSB, NS, Nightjet, OUIGO, Queensland and many more), plus 11 hardware-vendor drivers. The remediation must therefore be incremental and verified against the existing test suite, with per-train rollback, since these fleets carry passengers.

One urgent item: the device passwords are stored in clear text and committed into the source-code repository (finding S3, detailed in §B3). Anyone with access to the repository, the installable package, or a train's maintenance computer can read every device password. This should be remediated immediately — rotate the passwords and move them out of the repository — ahead of the rest of the programme.

2. What we reviewed and how

We reviewed the OBN code as R&D ships it (not our own field-patched copies), as a fresh, independent audit. The review ran in several phases:

- **Baseline:** size, complexity, dependency vulnerabilities, and test status, frozen for a consistent reference.
- **Architecture:** how the system is structured and where its weak points are.
- **Security & dependencies:** known vulnerabilities, injection surface, credentials, and the network-interface exposure.
- **Comprehensive bug sweep:** a systematic search for defects and edge cases across every subsystem, with each finding independently re-checked by a separate reviewer to remove false positives.
- Maintainability & tests, a per-finding severity assessment, and this report.

On verification quality: the audit ran in four independently-verified sweeps — the core bug sweep, a central-module sweep, a consumer-boundary sweep, and a device onboarding & prestaging sweep — each re-checked by a separate reviewer. Across all four, candidates were filtered hard: 21

were refuted on re-review and many more were downgraded in severity. The totals in §4.3 are the combined figure with a per-sweep breakdown, so nothing is double-counted and every defect traces to its section in Part B. The figures are filtered, not inflated.

3. The health of the code

By the standard objective measures, OBN is in better shape than the run of field incidents would suggest:

Measure	Result
Size	~12,000 lines of code; ~9,400 lines of tests (a healthy ratio)
Complexity	Average rates the top grade (A); every single file rates the top maintainability grade
Known vulnerabilities	None — automated CVE scan is clean
Injection	No unsafe shell use
Credentials	WEAK — device SNMP/SSH passwords are stored in clear text and committed to the repo (finding S3)
Error handling	Disciplined — no “swallow everything” blocks

Interpretation: this is a simple, clean codebase with a fragility problem. Simple code is the ideal candidate for targeted hardening — which is what the remediation plan (§6) does.

One important exception — credentials (findings S3–S7). The device SNMP and SSH passwords are stored in clear text and committed into the OBN code repository. A dedicated security check found the problem is broader than a single file: the same few passwords are reused across most device types (so one leaked password unlocks most of the fleet), they are shipped inside the installable package, they are left readable by any local user on every train’s jump-box, and they have sat in the repository’s history unchanged for years. There is also no automated check that would stop a new password being committed. This is the single most serious issue in the review. It is a process/configuration matter, not a flaw in the program’s logic, and it is fixable in place (rotate the passwords, move them to an un-tracked file, lock down file permissions, scrub history, add a secret scanner). It should be treated as urgent and actioned ahead of the rest of the programme.

4. Where the problems are

4.1 One systemic root cause

When the network does not answer a query (a routine event during device reboots), OBN quietly produces an empty result that later code uses without checking. That single design choice is the origin of the dominant class of failures — and it is also why the same kind of bug keeps reappearing in different places. **These are not unrelated bugs; they are one pattern repeated.** The onboarding review (§B10) showed this is the local form of a broader trait: across the whole path that brings a device on board, OBN never checks what it found against what it expected, and never stops to flag a mismatch — it enrolls whatever answers on the network, numbers whatever layout it discovers, pushes whatever the rules specify, and reports success regardless. So a mis-imaged train, a miscable, a coupled neighbour train, a stale snapshot or a changed vendor firmware (§B11) all surface as a quietly wrong result instead of a clear failure. The empty-result pattern above is one instance; the fix is the same in spirit — make the mismatch explicit and loud.

4.2 One real architectural weakness, and it is contained

The hardest piece of logic in OBN — working out the train’s layout from the network — is written separately per customer instead of once in a shared, tested component. The code holds 14 specialised report classes; the customers without a bespoke class reuse a generic one, but each still carries its own layout template (~49 fleets across 65 template repositories in total). That is why a fault found for one customer tends to exist, unnoticed, for several others. It is the one part of the design worth changing, and it is confined to a single layer.

4.3 What the audit found

Confirmed defects	Needs runtime confirmation	Refuted on review
48	45	21

The totals above combine four independently-run, separately-verified sweeps. They are presented as a grand total here and broken down by sweep below, so no figure is double-counted and each can be traced to its section in Part B. Across all four, the large majority of confirmed defects reduce to the recurring patterns described in §4.1 — the empty-result pattern and its broader fail-silent character — with most of the remainder being “silent success,” an operation reporting that it worked when it did not.

Breakdown by sweep:

Sweep	Section	Confirmed	Needs-runtime / Refuted
Core bug sweep	B4	15	24 / 8
Central-module (high-blast-radius)	B8	6	13 / 4
Consumer-boundary (NMS / Zabbix)	C3	8	7 / 3
Onboarding & prestaging	B10	19	1 / 6
Total		48	45 / 21

The most serious confirmed items across all four sweeps: a loop that can hang or exhaust memory during report generation on mis-cabled trains; an operation that reports a firmware or configuration push succeeded when the device never took it; the absence of any check that a device belongs to this train before its identity is written to it (so a coupled neighbour train can be mis-configured); and a maintenance interface that lets devices be rebooted over the network with no login required. A separate review of vendor firmware-coupling (§B11) additionally characterised seven ways a vendor firmware change can silently break device support; it is a characterisation of exposure rather than a defect count, so it is not added into the totals above.

Notably, **the audit independently rediscovered the same defects we have been patching in the field** — confirming both that the review is finding the real problems and that the existing fixes are on target.

5. Overall code-health assessment

A summary of the codebase across the dimensions that matter for the remediation work. Scores are 1–5, where 5 is healthy. They indicate where the code is strong (low effort to maintain) and where the remediation should concentrate.

Dimension	Score	Read
Structural complexity (simpler = easier to maintain)	5 / 5	Strong
Dependency & security health	4 / 5	Strong
Architectural soundness	3 / 5	Mixed
Test coverage & quality	3 / 5	Mixed
Defect density & pattern recurrence	2 / 5	Focus here

The two lower scores point directly at the remediation priorities: the recurring defect patterns (§4.1) and the report-layer architecture (§4.2). The strength scores (complexity, dependencies, security) confirm the rest of the codebase is a sound base to build on — the recurrence has a **single common fix**, so one pattern fix clears many defects at once.

6. Remediation plan

Seven workstreams. The first is urgent and should be actioned immediately; the rest are ordered by leverage. Each is independently shippable and checkable against the existing test suite, so the work can land incrementally with per-train rollback.

1. **Get the credentials out of the repository — urgent, do now.** Move the device passwords to an un-tracked secrets file, change (rotate) the exposed passwords on the devices, and remove them from the repository's history.
2. **Fix the root cause.** Make “no answer from the network” an explicit, checked condition so these failures are caught when the code is built, not on a train in service.
3. **Verify before reporting success.** Replace every “assume it worked” with a confirmed-success check on firmware, configuration, and reboot operations.
4. **Consolidate the five weakest layout handlers onto the engine we already have.** A safe, well-structured train-layout engine already exists in the code (the one most fleets use). Five customer-specific versions were hand-written less safely than it — move those five onto the existing engine, starting with the two that can currently crash or hang report generation. This is the single architectural improvement, and because the good design already exists it is consolidation, not invention — smaller and lower-risk than building something new.
5. **Secure and test the maintenance interface.** Add access control to the network interface that can reboot devices, and add tests to the few critical areas that currently have none.
6. **Check a device belongs to this train before configuring it.** Today OBN trusts any device that answers on the network and will stamp this train's identity onto it — so a neighbouring coupled train, a mis-cabled unit, or a mis-imaged spare can be (mis)configured as one of ours. Add a per-train device roster and a membership check before a device is enrolled and before its configuration is written. This one change closes the whole onboarding trust gap (§B10) and contains the cross-train mis-configuration risk.
7. **Stop hand-patching trains.** Route the field fixes we have already proven through the normal build-and-release pipeline so every train benefits.

PART B — TECHNICAL FINDINGS

Audience: engineering. Every claim below is cited to file:line under src/usr/share/obn/ and backed by the working artifacts in findings/obn_platform_review_2026-06-27/ (radon output, pip-audit, the adversarially-verified findings register, and per-phase summaries). Baseline: origin/master v2.3.12 (8042c8d), analysed in a detached read-only worktree; local fix branches were not used.

B1. Scope, method, baseline metrics

- **Scope:** whole platform (14 report classes in the app, backing ≈49 customer fleets across 65 template repos — verified live in GitLab — and all 11 vendor drivers), deepest where field evidence exists (DOSTO: vdsrail, report_dosto_neu, snmpdevice, tree).
- **Method:** baseline freeze → architecture trace → dependency/security audit → comprehensive bug sweep with two-stage independent (adversarial) verification → maintainability scoring → this report.

Metric	Value
Source	11,974 LOC / 76 files
Tests	9,440 LOC / 53 files (1.27:1)
Avg cyclomatic complexity	A (3.81) — 503 A / 78 B / 14 C / 7 D / 3 E / 3 F
Maintainability index	Every file grade A (worst: report.py 34, snmpdevice.py 37)
Dependencies	25 pinned; pip-audit: no known vulnerabilities
Typing	213/515 functions annotated (41%); no mypy in CI
Broad / bare excepts	6 (all documented) / 0

Platform scope (verified live in GitLab, VPN, 27 Jun 2026): the onboard GitLab group holds 290 projects, of which 65 are OBN train-layout template repositories spanning ≈49 customer fleets (TGVM alone has 10 layout variants, DOSTO 6). The OBN app itself ships 14 report classes — specialised topology handlers; fleets without a bespoke class reuse GenericReport plus their own template (e.g. Nightjet declares report_module: GenericReport). So the 14 classes are dispatch handlers, and the real deployed scope is the ≈49 fleets / 65 templates. Review baseline re-confirmed current: origin/master is still 8042c8d / v2.3.12 (latest tag 2.3.12).

B2. Architecture

OBN is cleanly layered. The diagram below shows command dispatch, the core subsystems, the device-class hierarchy, and the published outputs to downstream consumers:

OBN architecture — command dispatch and the device/report layers

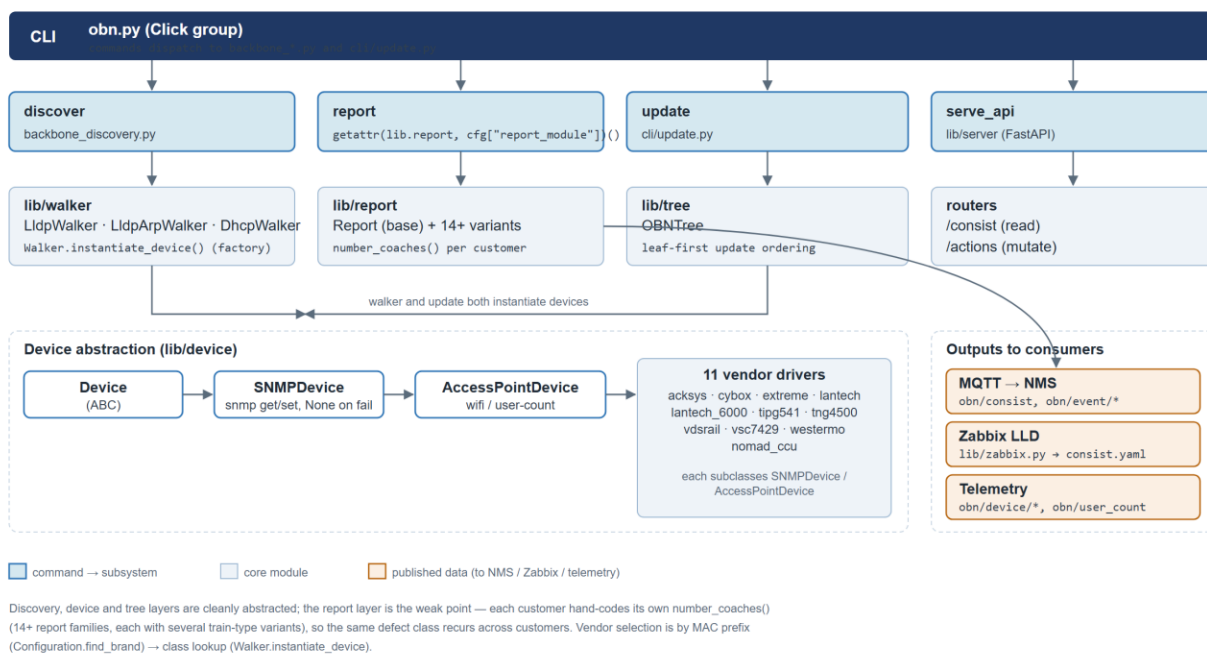


Figure 1 — OBN architecture. Vendor selection is by MAC prefix (Configuration.find_brand) → class lookup (Walker.instantiate_device); both discover and update instantiate devices through this factory.

The discovery, device-abstraction and tree layers are reasonably abstracted: a vendor is selected by MAC prefix (Configuration.find_brand, configuration.py:174) and instantiated by name (walker.py:64). Adding a vendor or customer means writing a class and editing an `__init__.py` — moderate friction, not broken.

B2.1 The architectural weakness — the report layer

Report.number_coaches is declared @abstractmethod (report.py:237) and all 15 report modules (base + 14 customer classes) define their own implementation — and several branch further per train-type (DOSTO: nv4/nv6/fv5/fv6; TGV: TGV/Dasye/RBi; CCJPA: WD1/WD2). The 14 classes are the specialised handlers; the broader fleet (~49 customers / 65 template repos, verified live in GitLab) reuses GenericReport plus a per-customer template, so the layout diversity this logic must satisfy is far larger than 14. Each implementation is 90–150 LOC of bespoke topology logic; only a ~5-line queue-init prologue is shared.

But the problem is narrower than “all 14 are bad.” GenericReport — the class ~30 of the ~49 fleets actually run on — is the safe reference: it re-enqueues only on a successful numbering (report_generic.py:95-96), None-checks every get_device result, guards its one .serial.startswith, and handles an unmatched connection by a topology-YAML lookup that logs and skips rather than crashing (fixed_consist_algo). We checked each bespoke class against that bar. Seven match or beat it (TGV, TGV2020, Queensland, CCJPA-WD1, CCJPA-WD2, DSB, Daisy). Five fall below it — and only two of those carry high-severity report-layer defects:

Report class	Loop	None-guarding	Worst report-layer defect
DostoNeu	VULNERABLE	partial	HIGH — unconditional re-enqueue (:158) → infinite loop / report OOM on malformed cabling
VIA	safe	UNGUARDED	HIGH — to_device.type deref with no None-check (:251); unbound box_device (:144) aborts the report
ACE	safe	partial	MED — master_switch deref unguarded (:73); port-10 operator-precedence mis-numbering (:123)
Dani	safe	partial	LOW
Luna	safe	partial	LOW
TGV / TGV2020 / Queensland	safe	guarded	none / low — at or above the Generic bar
CCJPA-WD1/WD2 / DSB / Daisy	safe	guarded	none / low — at or above the Generic bar

So the real architectural finding is not “the report layer is broken” — it is that **five bespoke per-customer classes (DOSTO, VIA, ACE, Dani, Luna) reinvented the numbering walk worse than the GenericReport engine that already exists in the same codebase**, with DOSTO and VIA the urgent two. Complexity concentrates in exactly these hand-rolled methods (VIAResult.number_coaches F-59, DostoNeuReport.number_coaches F-41) and that is where the field OOM bug lives. All 14 classes still inherit the report-agnostic findings (the normalise_devices drop, MQTT flap, compare_json staleness) covered elsewhere in this part.

B2.2 The systemic root cause — None on SNMP failure

_snmp_parse_results (snmpdevice.py:283) continues on any error and returns the accumulated dict (empty on total failure); _snmp_get then returns result.get(oid) -> None (snmpdevice.py:340). Device properties (serial_number, firmware_version, configuration_version, neighbours) propagate that None with no exception. Every consumer must defensively guard each dereference; where a guard is missing, you get a NoneType crash — and SNMP timeouts during the exact reboot windows these methods poll through are ordinary operating conditions. This single design decision is the origin of the dominant defect class.

B3. Dependencies & security

- **CVEs:** pip-audit reports no known vulnerabilities across all pinned runtime + dev deps. Versions current (FastAPI 0.137, cryptography 48, paramiko 5).
- **Injection surface: clean.** Only 2 subprocess calls (lldp.py:23, lldparpwalker.py:157), both list-form with shell=False; no os.system; jinja2 uses StrictUndefined on operator config. No hard-coded credentials in the Python source — but credentials ARE committed in plaintext in the vendors.yaml config (finding S3 below).

B3.0 Findings S3–S7 (HIGH) — credential exposure cluster (plaintext, reused, shipped, world-readable, unrotated)

OBN reads device credentials as plaintext from config — `snmp_user`, `snmp_password` (used for BOTH the auth AND priv passphrases, `snmpdevice.py:165–167`) and `snmp_v2c_community` (`snmpdevice.py:158`). There is no decryption / keyring / vault layer anywhere in OBN.

The live credentials are committed to the OBN git repository in `src/etc/obn/vendors.yaml` (present in HEAD since commit 5429a28): 12 distinct `snmp_password` values, 2 `ssh_password`, and a `v2c_community` string in clear text — the real production secrets (e.g. `NomadStayOut!`, `NomadComelN`, `DelicateSoundofThunder`, `SW_V1A_R0`). Anyone with repo read access, a copy of the `nd-obn .deb`, or read access to `/etc/obn` on any CCU obtains every device credential fleet-wide.

A safer mechanism exists but is not enforced: `secret.yaml` (`snmpdevice.py:132` — “Some projects use non-default uname and pw, if so: find them in `secret.yaml`”) can override per-brand creds, but it is optional, `vendors.yaml` still ships live passwords as the default, and `secret` is not even in `.gitignore`. The insecure default is what ships.

Severity: HIGH — a fleet-wide credential exposure baked into version control, more serious than S1 (which depends on network exposure). A follow-up targeted security/config sweep found the root finding S3 is amplified by four further issues:

- **S4 — password reuse across the fleet.** “`NomadStayOut!`” is shared across 8 vendor blocks and “`NomadComelN`” across 3; one leaked string unlocks SNMP write on most devices (incl. the `vdsrail` consist switches this project commissions). These are not read-only — the same creds drive the `config/firmware/reboot` OIDs.
- **S5 — secrets shipped in the published package.** `.gitlab-ci.yml:77` (`fpm ... src=/`) bakes `/etc/obn/*.yaml` verbatim into the `nd-obn .deb` pushed to the registry — a wider audience than repository read access.
- **S6 — world-readable on every CCU.** `packaging/postinst.sh` `chmods` only scripts, never `/etc/obn/*.yaml`, so the config installs at mode 644 — readable by any local unprivileged user; and the `serve-api/telemetry` services run as `User=root`.
- **S7 — never rotated, still in history.** The secrets were committed years ago and across 64 commits to the file have never been removed or rotated; `git log -S` still recovers them. The intended `secret.yaml` override is dead — the `vendors.yaml` header explicitly forbids changing the file, so the cleartext defaults ARE the production credentials.

Guardrail gap: no secret scanner exists anywhere (pre-commit runs `bandit` only; CI runs `pip-audit` only), and `.gitignore` has zero credential patterns — a committed secret passes every gate.

Remediation (no external action taken in this review): rotate the exposed device passwords (and stop reusing one string fleet-wide); move secrets to an un-tracked `secret.yaml` and relax the “do not change” header so the override is usable; add `secret.yaml/*.env/key` patterns to `.gitignore`; replace committed values with placeholders; `chmod 600` the config in `postinst`; scrub git history; and add a secret scanner (`gitleaks/trufflehog/detect-secrets`) to pre-commit AND CI so this cannot recur.

Surfaces checked and clean: `vendors_extra.yaml`, `backbone-discovery.yaml`, `coach_ap_mappings.yaml`, `topology.yaml`, `docs/`, `systemd` units, `tools/`. This whole cluster is `process/config/packaging`, not program logic; it is fixable in place but urgent.

- **Error handling: disciplined.** All 6 broad except `Exception` are documented; 0 bare `excepts`.

B3.1 Finding S1 (MED–HIGH) — unauthenticated, GET-mutating REST API

`lib/server` mounts `/consist` and `/actions` with no authentication (no API key/JWT/Depends guard). `routers/actions.py` exposes hardware-affecting operations as HTTP GET: `GET /reset_device/{ip} (:20)`, `GET /togglepoe/{ap_ip}/{state} (:83)`, `GET /getlog/{ip} (:53)`. `reset_device` reboots a device on a live train. Severity is exposure-dependent (localhost bind vs management-VLAN reachable), but the code offers no defense-in-depth regardless. Fix is small: add an auth dependency and switch mutations to POST.

S2 (LOW) — weak SNMP encryption defaults, and other lesser flags: the default SNMP profile uses dated algorithms (MD5 authentication + DES privacy, snmpdevice.py:143; the stronger AES is available but not the default — this compounds S3, since the same plaintext password is reused for both); the legacy, weaker SNMPv2c mode is still permitted; and the httpx2/httpcore2 low-profile dependency forks (cli/update.py:15, ospf.py:35) warrant a provenance/maintenance review.

B4. Bug & edge-case sweep (adversarially verified)

47 raw → 15 confirmed, 24 needs-runtime-confirmation, 8 refuted, 0 duplicate. Defect classes:

Class	Confirmed	Needs-rt	Meaning
none_deref	5	18	unguarded consumer of an SNMP-derived None
error_masking	5	0	real failure reported as success / 500
silent_noop	4	2	reports success without doing the work
regex_assumption	0	3	parser assumes a device response format
unbounded_loop	1	0	number_coaches topology walk with no visited-set → can loop forever
other	0	1	miscellaneous (timing/ordering)
Total	15	24	(plus 8 refuted = 47 candidates)

B4.1 Confirmed findings (15)

Sev	Location	Class	Finding
HIGH	report_dosto_neu.py:158	unbounded_loop	number_coaches() re-enqueues with no visited set -> infinite loop / OOM on malformed cabling
HIGH	tree.py:24	none_deref	root selection ...startswith('box1'))[0] crashes on None serial or empty list (untested module)
HIGH	vdsrail.py:74	none_deref	firmware-poll re.search("Not running", None) crashes in the reboot window
HIGH	actions.py:5	error_masking	HTTPException imported from http.client not fastapi -> every 404/405 becomes a TypeError
HIGH	actions.py:48	error_masking	reset_device collapses not-found / wrong-type / real-failure into {success:False} HTTP 200

Sev	Location	Class	Finding
MED	vdsrail.py:96	silent_noop	firmware/config/reboot return True unverified (same pattern in lantech/tng4500/westermo)
MED	report_via.py:144	none_deref	box_device referenced unbound when no CCU serial matches -> aborts VIA report
MED	actions.py:112	none_deref	togglepoe derefs parentSW.mac when cache seeds port but loop nulls parentSW
MED	actions.py:57	error_masking	getlog 500s and leaks the report path on missing report / unknown IP
MED	actions.py:25	silent_noop	action endpoints use an @lru_cache'd device list -> stale consist after a re-report
MED	backbone_handle_ssid.py:57	none_deref	continues after failed report load -> filter(None) TypeError
MED	cli/update.py:276	error_masking	empty target -> all([])==True -> misleading "readonly devices" success
LOW	acksys.py:420	error_masking	raise AcksysDevice(...) (not an Exception) masks the real error
LOW	cli/update.py:276	silent_noop	no-op success on mistyped/stale IP (companion to above)
LOW	backbone_handle_ssid.py:29	silent_noop	load_target_ssid is a TODO stub hardcoded to "Waffles" (unfinished command)

B4.2 The defects are two systemic patterns, not 39 unrelated bugs

- **23 of the 39 actionable defects are the None-dereference class** (5 confirmed + 18 needs-runtime) — every one a consumer of an SNMP-derived value used without a guard. The two long-standing field bugs (device.py:66/72, needs_*_update None.endswith) were independently rediscovered by the sweep.
- The rest are silent-success / error-masking — the return True # TODO family (vdsrail.py:96,125,131; lantech.py:67,83,87; replicated in tng4500/westermo) and the broken actions.py error paths.

B4.3 Whole-platform spread (not DOSTO-only)

The initial sweep flagged the number_coaches defect family as candidates in VIA, ACE, DSB, Dani, Queensland and rules_engine. The per-class verification against the GenericReport bar (see §B2.1) then confirmed five classes genuinely below the bar — DOSTO, VIA, ACE, Dani, Luna — while DSB and Queensland were re-checked and found to meet the bar (their candidate flags did not survive verification). This per-customer hand-rolling is still the architectural weakness: the same defect class recurs across the bespoke classes, where GenericReport (and the seven bespoke classes at or above its bar) is not affected.

B4.4 The untested-critical-path payoff

tree.py (update-ordering; zero tests) returned 3 high-severity crashes (root-None-serial at :24, empty-list [0], unresolved-neighbour None.type at :37). Direct evidence that the test gaps ship bugs.

B4.5 Verification credibility

The adversarial pass refuted 8 raw findings (e.g. a claimed operator-precedence bug in ACE and a claimed min(int, None) in CCJPA, both already guarded) and downgraded several severities — confirming the register is filtered, not inflated.

B5. Maintainability & tests

- **Complexity is low** (avg CC 3.8; all files MI grade A) — a simple codebase, cheap to harden.
- **Coverage is uneven but sensibly weighted:** report layer >1:1 and green (280 pass); but tree.py, actions.py, tipg541.py, tng4500.py are untested, and there is no mypy despite None-propagation being the dominant bug class.
- **Self-documented debt:** the # TODO: return True only on successful call stubs are the silent-success bug class admitted in comments.
- **Bug-class recurrence has a common fix** — one pattern fix (typed optionals + guard helper) retires dozens of latent defects at once.

B6. Code-health scorecard

Each dimension scored 1–5 (5 = healthy) with the cited evidence. The two low scores are the remediation priorities; the high scores are the sound base the work builds on.

Dimension	Score	Basis
Structural complexity	5	avg CC 3.8; all files MI grade A
Dependency / security	3	pip-audit clean; injection-clean; minus S3 (plaintext creds in repo, HIGH) and S1
Architectural soundness	3	clean except the report layer (5 of 14 classes below the GenericReport bar)
Test coverage	3	good on reports; gaps in tree/actions/2 drivers
Defect density & recurrence	2	39 actionable defects but a small number of common-fix patterns

The remediation plan (§B7) targets the two low-scoring dimensions directly: the recurring defect patterns and the report-layer architecture. Each fix is bounded and checkable against the existing test suite, so the work lands incrementally with per-train rollback.

B7. Remediation programme (technical)

1. **Typed SNMP results + mypy in CI** — catch the none_deref class at build time, not on a train.
2. **Verify-after-set helper** — replace every return True # TODO with a confirmed-success check.
3. **Consolidate the 5 below-bar report classes onto the existing GenericReport pattern** (report_generic.py fixed_consist_algo + topology-YAML — already safe: guarded enqueue, None-checked get_device, log-and-skip on unmatched topology). Migrate DOSTO and VIA first (HIGH), then ACE, Dani, Luna. The other 7 bespoke classes already meet the bar. The good engine exists in-tree — this is consolidation of 5 classes, not a new build, and not all 14.

4. **Remediate S3 credentials (urgent)** — move secrets to an un-tracked secret.yaml (mechanism already exists, snmpdevice.py:132), add it to .gitignore, replace committed values in vendors.yaml with placeholders, rotate the exposed device passwords, scrub git history.
5. **Fix the API** (auth + POST mutations + correct HTTPException) and backfill tests for tree.py, actions.py, and the two untested drivers.
6. **Keep field fixes flowing upstream** via GitLab → deb → Puppet rather than hand-patching CCUs.

B8. Central-module (high-blast-radius) audit

A late pass audited the shared/base/factory/gate modules that earlier sweeps had cited as context but never targeted as first-class subjects — the same gap that initially hid the GenericReport assessment. These run platform-wide (one bug = every device / every train), so they were re-checked deliberately, with adversarial verification. Result: 23 raw → 6 confirmed, 13 needs-runtime, 4 refuted. The core is broadly careful; the verifiers refuted four overstatements and narrowed most “fleet-wide” claims. No new high-severity platform-wide bug — but three things are worth acting on:

1. **“obn validate” is a weak commissioning gate.** It always exits 0 (failures never reach the exit code), makes no positive structural assertion (a non-empty but wrong/short device list reads as healthy), accepts present-but-None fields, and the only real topology checks (a TGV subclass) are never wired in. So a broken or short consist can validate as “OK”. The team runs this during commissioning — do not treat a green validate (or its exit code) as proof a train is correct. (validate.py:47/136/190, backbone_validate.py:146.)
2. **The on-disk device cache is dead code hiding two armed traps.** The disk cache silently does nothing today (its directory is never created, so it fails safe to live SNMP), but it conceals a key-collision (devices built with the default MAC share one cache slot — one device’s serial/firmware served to another) and empty-string poisoning that would activate the moment someone “fixes” the cache path. A trap for a future maintainer. (decorators.py:48/75/101.)
3. **Same systemic patterns reach the centre.** The remaining items are the None/empty-string and stale-read classes from Part B, now seen in the base layer: generic_get returns “” (not None) on a walk-miss → a telemetry crash for the one walk-preferring AP brand; a failed SNMP read is cached for the whole process; a shared-IP device (the documented ZFR/Sprechstelle redundancy pair) is silently dropped from discovery by an AND-gate short-circuit. All in-place fixes. (snmpdevice.py:278/234, walker.py:24.)

Bearing on the remediation: every item here is a contained, in-place fix, and they are the same two systemic patterns already identified, now shown to reach the central modules — so they fold into the same workstreams. The value of this pass is methodological: every high-blast-radius component has now been audited as a target, not assumed clean because it is “infrastructure.”

B10. Onboarding & prestaging fragility audit

Why this section exists. Product reported that vendor firmware updates can break OBN’s hardcoded device support. That is a real concern, addressed in B11. It also raised a sharper question: is firmware change the only way OBN’s device-onboarding model fails, or one instance of something systemic? This section answers that. It audits the device onboarding path (how OBN recognises and manages a device) and the prestaging preconditions (the CCU-image, network and human state OBN assumes but never establishes or verifies), deliberately excluding firmware-string drift so the two themes do not double-count.

Method. Two multi-agent sweeps across six onboarding/prestaging surfaces, every finding adversarially verified against the source on origin/master. Twenty-six raw candidates reduced to nineteen confirmed, one needs-runtime, and six refuted. The verifiers refuted six over-statements (including two reassuring ones: a fallback guard already recovers the redundant-path device-drop

case, and the access-point config path already guards its SNMP result) and down-scoped several severities. Full evidence with file-and-line anchors: findings/obn_platform_review_2026-06-27/ONBOARD_fragility_register.json.

B10.1 The single root cause

OBN's onboarding/prestaging path has no expectation-versus-reality reconciliation and no fail-loud seam. It enrolls whatever answers on the network, numbers whatever the topology walk discovers, pushes whatever the rules and templates specify, and reports success regardless. Every wrong precondition — a mis-imaged CCU, a miscable, a foreign or coupled neighbour consist, a stale snapshot, a transient SNMP miss, a missing rule — therefore surfaces as a silent wrong result rather than a loud stop. Firmware drift (B11) is the same defect on the vendor-version surface.

B10.2 Confirmed findings (19)

Push safety — OBN reports success when nothing happened.

Severity	Location	Finding
HIGH	vdsrail.py:95/124/130	set_firmware / set_config / reboot all return True (marked TODO) without verifying the SNMP SET took
MED	westermo.py:76-83	AP firmware push returns True on the SET echo — no completion poll, no reboot, no post-flash verify (partial-flash trap)
MED	vdsrail.py:74/116	poll loop runs a regex on a possibly-empty SNMP result in the reboot window — uncaught error mid-batch (loud)
MED	update.py:276-285	empty target set exits as a readonly-devices success — a mistyped or stale push IP reads as a benign no-op

State and persistence — a transient miss poisons the snapshot.

Severity	Location	Finding
MED	snmpdevice.py:226/234; decorators.py:36	a transient SNMP miss caches an empty firmware/config value and writes it into discovery.json; only a fresh discover clears it
MED	report/device.py:23-25; report.py:130	compare_json excludes the coach/device number, so a corrected topology re-numbering never republishes to the network-management system

Security and trust boundary — OBN trusts whatever answers.

Severity	Location	Finding
MED	configuration.py:174; walker.py:38	device identity is the MAC vendor-prefix only — anything answering with a known prefix is onboarded as a trusted switch or AP
MED	report_dosto_neu.py:200-264	a coupled neighbour consist's switches and APs are enrolled over the coupler and fallback-numbered as ours (field-observed)
MED	update.py:84; snmpdevice.py:488	no does-this-device-belong-to-this-train gate before stamping this train's identity config onto it (the Fzg 133 cascade, cross-train)

MED	actions.py:25-34; vdsrail.py:129	the action API reset/PoE acts on the stale report IP-to-MAC binding (two-minute leases) with no live identity re-check
-----	-------------------------------------	--

Topology and coach numbering.

Severity	Location	Finding
HIGH	report_dosto_neu.py:61/158	the numbering walk has no visited set; unnumbered mutually-adjacent switches re-enqueue without bound — obn report hangs and exhausts memory (the field bug-10 class)
MED	report_dosto_neu.py:42-43	numbers whatever the walk discovers with no assertion against the expected consist; sibling projects have the structural count-check this path lacks
MED	report_dosto_neu.py:88-155	mis-numbers or silently drops primary-path miscabled switches; the unexpected-connection report wired into seven sibling projects is never called here
MED	report_dosto_neu.py:42	silently seeds the CCU into the wrong coach when the train type is unset — the whole consist shifts by one (the generic path hard-exits here; this one does not)

Prestage preconditions — assumed-but-unverified CCU, network, and human state.

Severity	Location	Finding
MED	update.py:271	update/validate consume the previous discovery snapshot with no discover-before-report ordering or staleness guard
MED	report.py:306-318	a device slot with no matching rules entry gets an empty target and is silently dropped from the update set
MED	snmpdevice.py:488; backbone_validate.py:42-49	an unvalidated train identifier is stamped into every config; validate self-derives from the same value, hiding a mis-image
MED	snmpdevice.py:246-251	the TFTP firewall helper crashes (rather than returning gracefully) on a missing ipset, aborting the update outside its error handler (loud)
MED	ccudevice.py:61	the CCU's own IP is derived arithmetically from config and never reconciled against any interface
LOW	validate.py:177-190	the completeness gate checks key-presence only and never checks that a rules target exists — false consist-complete confidence

B10.3 The fix converges to three primitives

Every confirmed finding maps to one of three in-place hardening primitives — no rewrite, consistent with the IMPROVE assessment:

4. **P1 — Closed-loop verification on every push.** Read back the device's actual config, firmware and reboot state after the SNMP trigger; never return an unverified success. Kills the false-success cluster (and half of B11).
5. **P2 — Per-train device roster and membership gate before enrol and push.** Cross-check each discovered serial, MAC and expected position against an expected roster; reject or quarantine non-members. Kills the entire security-trust cluster and the coupled-consist mis-config, and bounds the topology mis-number blast radius.
6. **P3 — Fail loud on precondition or expectation mismatch.** Assert the discovered consist against the expected layout; sanity-check the train identifier and CCU IP; assert a rules target exists per device; warn on a stale snapshot. Covers the topology and prestage clusters.

B10.4 Bearing on the review

7. **Verdict unchanged: IMPROVE.** Nothing here is a rewrite trigger; all are in-place fixes.
8. These are the same systemic classes the report already names (empty-value handling, silent success, validation gaps, stale cache) — now shown to extend across the whole onboarding and prestaging path, not just the report layer. Reinforces, does not overturn.
9. Two findings were independently confirmed by the earlier central-module sweep (B8) — cross-validation that this pass is sound. The six refutations and the severity down-scopes are the credibility check: the verifiers killed over-statements rather than rubber-stamping.

B11. Vendor firmware-coupling fragility

Origin. This section addresses the product complaint directly: a vendor firmware update can change the very information OBN has hardcoded to recognise and manage that device, breaking support silently. It is the vendor-version surface of the B10 root cause. Full evidence: [findings/obn_platform_review_2026-06-27/FW_DRIFT_register.json](#).

The coupling. OBN's device-support layer is pinned to a single firmware snapshot per device. Recognition (MAC vendor-prefix, model and revision strings), access (SNMP OIDs, auth protocol, credentials) and parsing (version-string regex, MIB tree shape, radio-band ordering) are all hardcoded in vendors.yaml and the per-vendor device classes. There is no firmware-version-aware logic anywhere in the device layer and no drift detection — a vendor firmware change has no seam to absorb it and fails silently at runtime. The code admits the recurrence twice in its own comments (the access-point revision map "requires regular TLC" because the vendor "changes the revision without informing Nomad"; switch error codes are "prone to change between firmware versions").

B11.1 Where the coupling lives

Sev	Surface	What a firmware update can change	Failure mode
HIGH	Version-string regex	format of the firmware/status string	regex returns nothing — silent "unknown" or a crash
HIGH	Model/revision maps	a new hardware or model string on a firmware rev	unknown model — wrong or no driver behaviour, silent
MED	MIB structure / encoded OIDs	OID tree, index depth, encoded command OIDs	SNMP read returns nothing or the wrong node — silent

MED	Band / radio ordering	adding 6 GHz, reordering the radios	wrong band counted or configured
MED	CLI output parsing	the CLI text format (firmware often re-words it)	parse miss — empty result
MED	SNMP auth / protocol	v2c to v3, weaker to stronger crypto, cred change	auth failure — the device "disappears"
MED	MAC vendor prefixes	a new prefix block on a new hardware batch	the device is skipped entirely from discovery

Two facts make this systemic, not a list of bugs. First, there is zero firmware-version-aware logic in the device layer — no version comparison, no conditional, no fallback — so a firmware change has nothing to absorb it. Second, the tests pin the snapshot but cannot detect drift: they assert known model strings and version formats, so new firmware that ships a new string passes the test suite and breaks in the field. That is the worst quadrant — green tests, broken train.

B11.2 Remediation

In effort order; all but the last are in-place hardening consistent with the IMPROVE assessment:

10. **Drift detection in the build pipeline (highest leverage, cheapest).** A scheduled job that reads a reference device per vendor (or a captured MIB snapshot) and asserts the hardcoded OIDs and model strings still resolve. Turns a silent field failure into a build alert before a train is in front of an engineer. No production change.
11. **Defensive parsing with an explicit "unrecognized" state.** Every version and model parser, on a miss, returns a typed unrecognized value that propagates to validate and report as an explicit flag — never a silent empty value. Folds into the empty-value-handling workstream.
12. **Fallback chains for the model and revision maps.** A documented "unknown revision — safest default plus warning" path instead of dropping the device.
13. **A full per-vendor capability contract test.** Capture the whole expected OID surface per vendor, not just the few values asserted today, so drift in any hardcoded value fails the build.
14. **Externalise the snapshot (strategic, larger).** Vendor support is hardcoded imports plus a do-not-edit config file, so every drift is a core code change and a package release. A versioned vendor-capability descriptor, keyed by firmware range, would decouple an OBN release from vendor-firmware support. A roadmap item, not a quick fix.

Bearing on the review. The complaint is legitimate but addresses a maintenance-process and defensive-coding gap, not a structural reason to replace OBN — every remediation above except the optional last is in-place hardening. Verdict unchanged: IMPROVE.

B9. Caveats & artifacts

- Device-vendor/walker/CLI tests could not be executed in the local Windows venv (a third-party isc_dhcp_leases packaging bug, not an OBN defect); they pass in CI on Debian. Report-layer + validate tests ran green locally (280 passed). Findings in the device chain are static-analysis-based unless marked otherwise; 24 findings are explicitly classed needs-runtime-confirmation.
- The review is of origin/master v2.3.12 as of 2026-06-27; subsequent upstream commits are not included.

Supporting artifacts: findings/obn_platform_review_2026-06-27/ — baseline/ (radon CC/MI/raw, pip-audit, dependency list, pytest baseline), P1_architecture.md, P2_dependency_security.md, P3_bug_sweep_summary.md, P3_findings_register.json (full verified register), P4_maintainability.md, P5_decision_model.md (per-dimension scoring evidence),

C3_consumer_boundary_summary.md, C3_consumer_findings_register.json, S_security_config_sweep.md, fleet_scope_gitlab_verified.md, onboard_group_inventory.json, generic_report_bug_applicability.md, bespoke_vs_generic_summary.md, bespoke_vs_generic_register.json, BS_blindspot_summary.md, BS_blindspot_register.json.

- **Coverage note:** after the report-layer, consumer, security and per-class passes, a dedicated central-module audit (B8) closed the last coverage gap — every high-blast-radius shared/base/factory/gate module has now been audited as a first-class target. Findings throughout are static-analysis-based unless marked needs-runtime-confirmation.

Scope of this part. OBN does not run in isolation — it publishes data that other applications consume. This part audits the OBN output surface on origin/master v2.3.12 as-is for edge cases and bugs whose blast radius lands in those consuming apps. It is a general audit; no single symptom frames it. Method: a 3-hunter sweep over OBN's output-producing code with independent adversarial verification (18 raw findings → 8 confirmed, 7 needs-runtime, 3 refuted).

C1. Who consumes OBN data, and how

Consumer	Fed by	Uses it for
NMS	OBN MQTT obn/consist + obn/event/device_* → MongoDB	Train diagram, device inventory (fw/config/coach), add/remove. Trusts OBN for “what devices exist.”
Zabbix	OBN Zabbix-LLD (lib/zabbix.py, from consist.yaml) for host discovery; its OWN SNMP/ICMP for up/down	Per-device/-port monitoring + alarms. Existence from OBN; liveness from its own polls.
Telemetry	OBN MQTT obn/device/stats, obn/device/ports, obn/user_count	Link/port stats, Wi-Fi user counts.

Key structural point: a device's existence comes from OBN, but its liveness (for Zabbix/NMS tiles) comes from independent polling. Divergence between the two is the root of “shows down / shows missing” confusion — and means some field-reported “OBN problems” actually live in the consumer or the device firmware, not in OBN.

C2. Three classes of consumer-impacting defect (origin/master as-is)

C2.1 Dropped devices (most serious)

normalise_devices() (report.py:254) rebuilds the device list keeping only devices with a non-None coach number, device number and type. **Any discovered, online device the topology walk could not fully number is silently deleted** — so it is absent from the NMS consist payload and from the Zabbix coach count. Triggers are general: a device on an unexpected port, a mis-cabled coupler, an AP not on its expected port, a switch the walk could not number, or a device reachable only via a redundant path the primary walk did not traverse. (The “NMS shows a switch down despite a redundant link” case is one instance of this class, not a separate bug.) Consumer symptom: NMS renders the device as missing/down though it is online; Zabbix never creates the host, so a real end-coach outage is invisible because absent is not the same as DOWN.

C2.2 Flap / spurious events (no debounce)

The MQTT add/remove diff (generate_device_mqtt_events, report.py:436/445) compares a single fresh discovery snapshot against the last published one, keyed purely by MAC, with **no confirmation or hysteresis**. One SNMP timeout in a cycle therefore emits device_removed (NMS

shows it gone) and a paired device_added next cycle — the device flaps in and out of the NMS inventory. This is the consumer-facing face of the same transient-None pattern found inside OBN in Part B.

C2.3 Stale-suppression and wrong fields

- **Stale suppression:** compare_json() decides “changed” from fields that deliberately EXCLUDE coach/device number and type. A corrected topology with other fields unchanged returns “not changed” and is never republished — NMS stays permanently stale on the old (wrong) numbering.
- **Wrong/stale fields pushed:** paintedCoachId is the literal “unknown” fleet-wide (OBN map file unset) → NMS and Zabbix painted-coach labels read “UNKNOWN”; firmware/config/serial “unknown” is sent as a real value; macAddr is UPPERCASE on the consist topic but lowercase on the event topic, risking MAC-correlation mismatch in NMS.
- **Template-driven LLD:** Zabbix LLD advertises the consist.yaml template grid, not what was actually discovered — extra real devices are never monitored, and template entries for absent devices become check-failures.

C3. Confirmed consumer-impacting findings (8)

Sev	Consumer	Location	Finding
HIGH	NMS	report.py:254	Unnumbered discovered devices deleted from consist payload → NMS shows device missing
HIGH	Zabbix	zabbix.py:44	number_of_coaches = max(coach_number) drops trailing coaches whose devices weren't numbered → end-coach outage invisible
MED	NMS	report.py:445	Dropped device also emits device_removed → false remove/add churn
MED	NMS	report.py:436	No hysteresis on add/remove diff — any one-cycle absence published immediately
MED	Zabbix	zabbix.py:57	LLD is template-driven not discovery-driven — real extra devices never monitored
LOW	NMS	report.py:339	paintedCoachId → “unknown” for a whole coach when its switch is unmapped/dropped
LOW	NMS	report.py:205	macAddr uppercased on consist, lowercase on events → MAC-correlation mismatch
LOW	Zabbix	zabbix.py:76	{#PAINTED_COACH_NUMBER} always “UNKNOWN” fleet-wide

Plus 7 needs-runtime-confirmation (single-SNMP-miss flap, fw/config/serial “unknown”-as-real, compare_json change-suppression, AP-with-coach-but-no-device-number drop, an unguarded coach_number key access in the LLD) and 3 refuted on review.

C4. Where each fix belongs

Layer	Issues
OBN-side	Dropped devices, add/remove flap, MAC casing, painted-coach plumbing, compare_json staleness, Zabbix LLD coach-count/grid — all in this review, all fixable in place.
Consumer-side	Phantom port-down (no debounce on Zabbix ifOperStatus), wrong template OIDs (already fixed 2026-06-08), PoE-fault not alarmed, NMS painted-coach toggle off — field-observed, NOT in OBN code.
Firmware-side	VDS SNMP AgentX subagent reinit and KMdev boot crash — the root of several “switch down” reports; a Stadler/R&D firmware matter.

Why this matters: several problems reported from the field as “OBN bugs” are actually consumer-side (Zabbix templates) or firmware-side (VDS SNMP agent) — **these fall outside OBN and need their own owners**. The genuine OBN-side consumer bugs are the same two root patterns as Part B (transient-None on a discovery miss → drop/flap; representation gaps), with the same common fixes. They add a small, well-scoped output-contract workstream to the remediation plan:

1. **Confirm-before-remove.** Require a device to be absent for N consecutive discovery cycles before emitting device_removed or dropping it from the consist — kills the flap class.
2. **Include-with-flag, don’t drop.** Emit an unnumbered-but-discovered device to consumers with an explicit “off-expected-wiring / unnumbered” flag instead of deleting it, so NMS shows it (flagged) rather than missing.
3. **Put identity in the change-compare.** Include coach/device number and type in compare_json so a corrected topology actually republishes.
4. **Tidy the field contract.** Consistent MAC casing across topics; distinguish “not read” from a real value; wire painted-coach end to end (OBN map + NMS toggle) if customer-facing coach numbers are wanted.

These add to the Part-B remediation programme as a contained output-contract workstream. The OBN-side consumer bugs are bounded and pattern-shaped (the same fixes as the rest of the programme); the remainder of the “OBN” pain reported from the field is owned by Zabbix and the switch firmware, and is tracked separately with those teams.